

开源软件基础

Git 与版本控制

Zhilei Ren



OSCAR Team, School of Software, Dalian University of Technology

December 8, 2017



Outline

1 Before Git

2 Git 入门

Vim

Why Vim

- 用于提交 commit 信息（非必要）
- 用于解决冲突（非必要）
- 那为什么？

编辑器效率对比

Classical learning
curves for some
common editors

Notepad



Pico



Visual Studio



vi

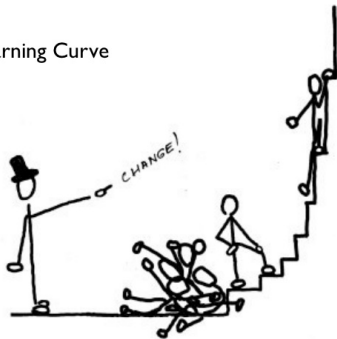


emacs



编辑器效率对比

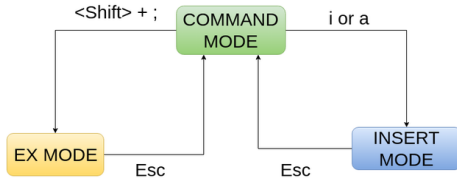
Learning Curve



高效编辑

- 基于模式
- 高效移动
- 命令组合
- 自定义/绑定

Vim 模式



高效移动



命令与文本对象

命令

- c: Change
- d: Delete
- y: Yank
- gU: Upper case
- gu: Lower case ...

文本对象

- aw: a word
- iw: inner word
- ab: a bracket
- ib: inner bracket
- at: a tag

合体（误）

命令 + 移动/文本对象

- **dj**: delete current and the next lines
- **y0**: copy to the line head
- **ciw**: change inner word
- **dab**: delete a bracket
- **gUit**: change inner tag to lower case ...

Practise makes perfect

继续学习

- vimtutor
- vimcasts^a
- 7 Habits For Effective Text Editing^b
- Maybe emacs^c

^a<http://vimcasts.org/>

^bhttp://v.youku.com/v_show/id_XMTIwNDY5MjY4.html?spm=a2h0k.8191407.0.0&from=s1.8-1-1.2

^c<https://www.bilibili.com/video/av6048518/?from=search&seid=12030402565005031700>

Outline

1 Before Git

2 Git 入门

Git

git (/t/) 是一个分布式版本控制软件，最初由林纳斯·托瓦兹 (Linus Torvalds) 创作，于 2005 年以 GPL 发布。最初目的是为更好地管理 Linux 内核开发而设计。

git 最初的开发动力来自于 BitKeeper 和 Monotone。很多著名的软件都使用 git 进行版本控制，其中包括 Linux 内核、X.Org 服务器和 OLPC 内核等项目的开发流程。

今天就能开始用的小知识

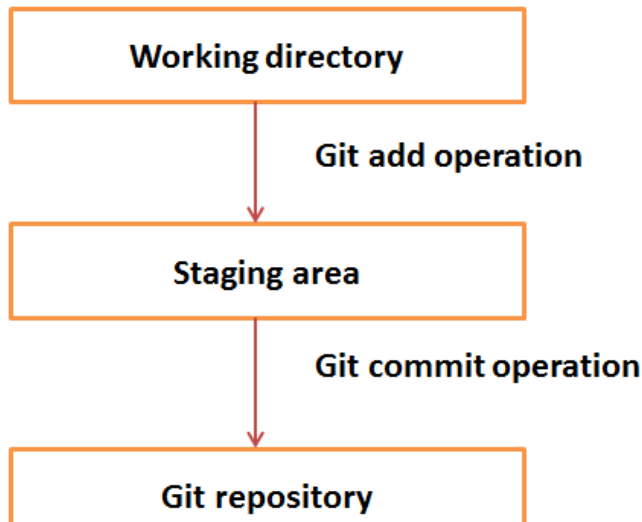
Git 名字的由来

林纳斯·托瓦兹自嘲地取了这个名字“git”，该词源自英国俚语，意思大约是“混账”。^a

I'm an egotistical bastard, and I name all my projects after myself. First Linux, now git.

^a<https://git.wiki.kernel.org/index.php/GitFaq>

Git 命令初步



Git 命令初步

```
#initialization
\$ git init

#configuration
$ git config --global user.name "Your name"
$ git config --global user.email "a@a.a"

# First commit
$ git add example.py

# adds file to the staging area
$ git commit -m "First commit"

# Second commit
$ git add another.py

# adds file to the staging area
$ git commit -m "Second commit"
```


Branching and Merging

- By default you are on the "master" branch
- To create a new branch and switch to it:
 - `$ git branch branchName`
 - `$ git checkout branchName`
- Merge changes back to master branch
 - `$ git checkout master`
 - `$ git merge branchName`
 - To delete branch
 - `$ git branch -d branchName`

My Typical Git Workflow

- Master branch is always clean and working
- When I want to try a new idea, create a branch
- Work and commit to that branch until code is clean and working
- Switch back to master and merge branch into master
- Delete branch

Things to Keep in Mind

- Any branch can be merged to any other branch
 - Careful, this might get confusing
- Must commit before switching branches
- Unless you add it, a file will not be tracked.
- If you want to ignore a file, add it to the `.git/info/exclude` file.
- To revert a file back to it's state at the last commit:
 - `git checkout fileName`
- To revert the entire repo to it's state at the last commit:
 - `git reset --hard HEAD`
 - CAREFULL! You'll loose all your changes since your last commit!
- <http://git-scm.com/>